CHAP APP DETAILED PROMPT

Create a feature-rich, single-page web application with Name [Ar Chats] a modern, dark, mobile-first design that simulates a complete chat and calling experience using HTML, CSS, JavaScript (ES6+), Firebase Realtime Database for backend, and WebRTC with Tone.js for real-time communication.

2. Technology Stack

HTML5

CSS3 (with CSS Variables and responsive design)

JavaScript (ES6+) - Modular design using type="module".

Firebase v9 Modular SDK (via CDN): auth, database.

Tone.js (via CDN): For generating the call ringtone.

WebRTC API: For real-time audio/video communication.

3. Key Features and Data Structures

A. Authentication & Profile

Auth: Implement Login and Sign Up using Firebase Authentication.

Profile Setup: After sign-up, prompt the user to set a name (max 20 chars), a bio (max 100 chars), and a profile identifier/emoji (max 2 chars, stored as emoji).

Unique ID: Generate a unique, public-

facing, four-digit string ID upon profile creation (e.g., ar-xxxx), stored as userId.

Profile Picture (dpUrl): The app must generate an SVG Data URL containing the user's emoji or text identifier. Do not implement file/storage upload. The dpUrl must be saved to the database.

Presence: Use Firebase .info/connected and onDisconnect to manage status/{uid} (online/offline, last_changed).

B. Contacts & Requests
Contact List (contacts/{uid}): Display a list of contacts. Listeners must fetch contact status and unread message counts.

Add Friend: Search for users by their

userId (e.g., ar-xxxx).

Friend Requests (requests/{targetUid}): Send and manage friend requests.

C. Chatting
Chat View: Show/hide main navigation and display a dedicated header/input area.

Typing Indicator: Implement a typing indicator (typing/{chatId}/{uid}) that clears after 3 seconds of inactivity.

Message Listener (messages/{chatId}): Listen for messages, display as bubbles (me/other). The chat scrolls to the bottom on new messages.

Unread Count (unreadCounts/{uid}/

{chatId}): Manage a count for unread messages. Reset to 0 upon entering a chat.

Chat Clear: Implement "Clear Chat" to set a clearedAt timestamp in userChats/{uid}/{chatId}. Messages older than this timestamp are filtered out of the display.

D. WebRTC Calls
Signaling (calls/{callId}): Use the Firebase Realtime Database for WebRTC signaling (offers, answers, ICE candidates).

Call Types: Support both Voice Call and Video Call buttons.

Call Flow:
Caller creates RTCPeerConnection.

Caller gets local media (getUserMedia).

Caller creates and sets local offer and pushes to calls/{callId}.

Callee listens for a call, accepts, gets media, sets remote offer, creates/sets local answer, and updates calls/{callId}/answer.

Both parties exchange ICE candidates via the database.

Call UI: A dedicated fullscreen modal (#call-screen) with localVideo (draggable) and remoteVideo. A simplified UI (#voice-call-ui) is used for voice calls.

Ringtone: Use Tone.js for an audio ringtone played during incoming calls, with a function to start/stop the loop.

Call Logging (callLogs/{uid}): Log calls with type (voice/video/missed), duration, and timestamp upon call conclusion.

Call Controls: Mute toggle and End Call button.

4. Visuals and Styling

A. Theming

Dark Mode: The entire app uses a dark theme.

CSS Variables: Must use the provided CSS variables:

--primary-color: #00A878; (Vibrant

Green)

--background-color: #121212;

--container-bg: #1e1e1e;

--text-color: #e0e0e0;

--bubble-bg-me: #00A878;

--bubble-text-me: #121212;

Font: Use Poppins (via Google Fonts CDN).

B. Mobile-First Layout
Container: A fixed-size container (max-width: 400px; height: 100vh; max-height: 800px;) simulating a smartphone screen.

Navigation: A clear bottom navigation bar with icons for Home, Requests, and Calls.

Side Menu: A slide-out menu (off-screen by default) for profile actions and logout.

Modals: All secondary interactions (Auth, Add Friend, Profile Edit, Call) use full-screen overlay modals.

C. DP Styling (Crucial)
The emojiToSVG(content) function is used universally.

The function takes max 2 characters of content and generates a black background SVG with centered white text/emoji.

NO image file inputs or storage logic should be present in the final code. All profile images must use the SVG Data URL generated from the user's saved emoji string.

5. Complete Code Structure

The entire application (HTML, CSS, and JavaScript) must be in a single index.html file.